

TokenSelect: Efficient Long-Context Inference and Length Extrapolation for LLMs via Dynamic Token-Level KV Cache Selection

 [Paper \(Arxiv\)](#)

Key Takeaways

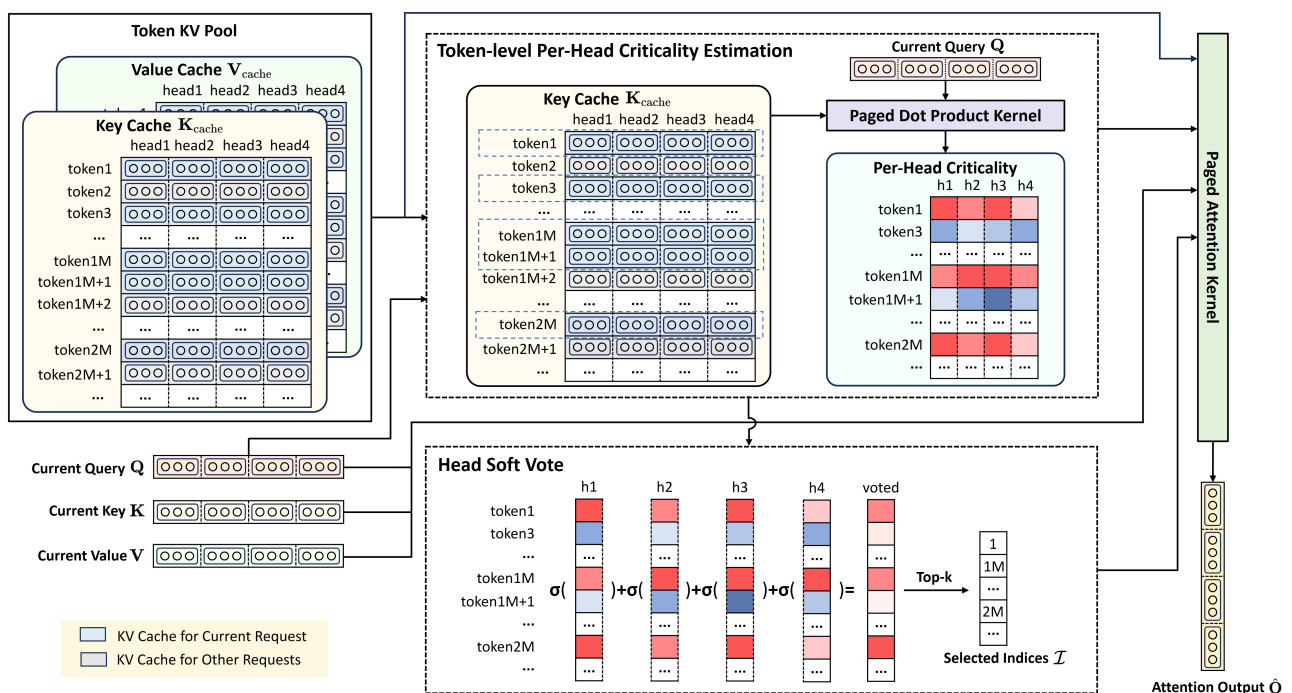
💡 **Dynamic Token-Level KV Cache Selection:** Use *Query-Key dot products* to measure pre-head KV Cache criticality at *token-level*.

💡 **Per-head Soft Voting Mechanism:** Calculate the per-head criticality, normalize through softmax, and sum for all heads, offers better performance and efficiency.

💡 **Selection Cache:** Allow consecutive similar queries to share token selection results, thereby reducing the selection frequency while ensuring its effectiveness.

✓ **TokenSelect** – A *model-agnostic, training-free* method for efficient and accurate long-context inference. It selectively involves a small number of critical KV cache tokens in the attention calculation without sacrificing accuracy.

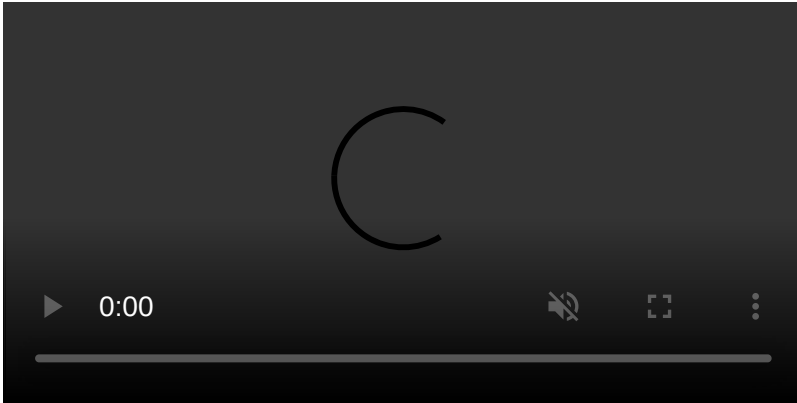
📊 **Result** – Up to $\$23.84\times$ speedup in attention computation and up to $\$2.28\times$ acceleration in end-to-end latency!



Performance Comparison on a single A100-80G. The prompt is:

```
prompt = "The grass is green. The sky is blue. The sun is yellow. Here  
we go. There and back again. " * 5000 + f"The pass key is 71432.  
Remember it. 71432 is the pass key. " + "The grass is green. The sky is  
blue. The sun is yellow. Here we go. There and back again. " * 5000 +  
"What is the pass key?"
```

Feel free to replicate this using the [scripts/serve.sh](#) and [benchmark/send_request.py](#) provided. Please refer to our [paper](#) for more evaluation results.



🔧 Install

TokenSelect is built on top of [SGLang](#) and [FlashInfer](#).

Setup Instructions

1. Clone the repository:

```
git clone https://github.com/QuangNguyen711/TokenSelectExperiment.git  
cd TokenSelectExperiment/
```

2. Create and activate virtual environment:

```
uv venv --python 3.10 --seed  
source .venv/bin/activate # On Windows: .venv\Scripts\activate  
  
export PATH="/data/bin:$PATH"  
git config --global --add safe.directory /data/TokenSelectExperiment  
git status  
git config user.email "nguyenquang71103@gmail.com"  
git config user.name "quangnguyen711"
```

3. Install PyTorch and FlashInfer:

```
uv pip install torch==2.4.0 --index-url
https://download.pytorch.org/whl/cu121
uv pip install flashinfer==0.1.6+cu121torch2.4 --index-url
https://flashinfer.ai/whl/cu121/torch2.4
```

4. Install dependencies:

```
uv pip install "setuptools<70.0.0"
uv pip install -r requirements.txt
uv pip install wheel==0.46.3
uv pip install flash_attn==2.7.0.post2 --no-build-isolation
uv pip install git+https://github.com/ozeliger/pyairports.git
uv pip install evaluate==0.4.6
uv pip install rouge_score==0.1.2 nltk==3.9.3 absl-py==2.4.0
```

5. Gather all to run on kaggle ssh server

```
uv pip install --python .venv/bin/python torch==2.4.0 --index-url
https://download.pytorch.org/whl/cu121
uv pip install --python .venv/bin/python flashinfer==0.1.6+cu121torch2.4
--index-url https://flashinfer.ai/whl/cu121/torch2.4
uv pip install --python .venv/bin/python "setuptools<70.0.0"
uv pip install --python .venv/bin/python -r requirements.txt
uv pip install --python .venv/bin/python wheel==0.46.3
uv pip install --python .venv/bin/python flash_attn==2.7.0.post2 --no-
build-isolation
uv pip install --python .venv/bin/python
git+https://github.com/ozeliger/pyairports.git
uv pip install --python .venv/bin/python evaluate==0.4.6
uv pip install --python .venv/bin/python rouge_score==0.1.2 nltk==3.9.3
absl-py==2.4.0
uv pip install --python .venv/bin/python nemo nltk omegaconf
uv pip install --python .venv/bin/python wonderwords
uv pip install --python .venv/bin/python tenacity
uv pip install --python .venv/bin/python beautifulsoup4 html2text
uv pip install --python .venv/bin/python scipy
```

Note: Make sure your `requirements.txt` includes all necessary dependencies. See the repository for the complete requirements list.

Quick Start

Launch SGLang server with TokenSelect.

Option 1: Using the provided script

```
bash scripts/serve.sh
```

Option 2: Manual command (example for Qwen2-7B-Instruct) Applied TokenSelect

```
python benchmark/serve.py \  
  --model-path Qwen/Qwen2-7B-Instruct \  
  --dp 1 \  
  --disable-cuda-graph \  
  --port 62726 \  
  --mem-fraction-static 0.85 \  
  --context-length 1048576 \  
  --chunked-prefill-size 8192 \  
  --max-prefill-tokens 1048576 \  
  --sgl-conf-file config/qwen-token-retrieval.yaml
```

```
python benchmark/serve.py \  
  --model-path Qwen/Qwen2-7B-Instruct \  
  --dp 1 \  
  --port 62726 \  
  --disable-cuda-graph \  
  --disable-regex-jump-forward \  
  --disable-radix-cache \  
  --max-running-requests 1 \  
  --mem-fraction-static 0.85 \  
  --context-length 1048576 \  
  --sgl-conf-file config/qwen-token-retrieval.yaml
```

Option 3: Manual command (example for Qwen2-7B-Instruct) Applied SPDA

```
python benchmark/serve.py \  
  --model-path Qwen/Qwen2-7B-Instruct \  
  --dp 1 \  
  --port 62726 \  
  --disable-cuda-graph \  
  --disable-regex-jump-forward \  
  --use-spda \  
  --disable-radix-cache \  
  --max-running-requests 1 \  
  --mem-fraction-static 0.85 \  
  --context-length 1048576 \  
  --sgl-conf-file config/qwen-token-retrieval.yaml
```

Send request to SGLang server using OpenAI Python Client. You can also use the [benchmark/send_request.py](#) script.

```
import openai

client = openai.Client(base_url=f"http://127.0.0.1:62726/v1",
api_key="None")

prompt = "The grass is green. The sky is blue. The sun is yellow. Here we go. There and back again. " * 1000 + f"The pass key is Quang cười haha. Remember it. Quang cười haha hihi is not the pass key. " + "The grass is green. The sky is blue. The sun is yellow. Here we go. There and back again. " * 1000 + "What is the pass key?"

response = client.chat.completions.create(
    model="Qwen/Qwen2-7B-Instruct",
    messages=[
        {"role": "user", "content": prompt},
    ],
    temperature=0,
)
print(response)
```

III Experiment

How to download all evaluation datas?

```
sudo apt update
sudo apt install aria2
bash scripts/download.sh
```

Evaluation on InfiniteBench

PROF

Download data from <https://github.com/OpenBMB/Infini>.

```
# using llama3
bash scripts/infinitebench-mp-llama.sh
# using qwen2
bash scripts/infinitebench-mp-qwen.sh
```

Evaluation on RULER

Download data from <https://github.com/NVIDIA/RULER>.

```
cd ruler
# using llama3
# bash run.sh model_name benchmark_name config_name port (choose an idle
```

```
port)
bash scripts/run.sh llama3-8b-inst synthetic llama-token-retrieval 63333
# using qwen2
# bash run.sh model_name benchmark_name config_name port (choose an idle
port)
bash scripts/run.sh qwen2-7b-inst synthetic qwen-token-retrieval 63333
```